5.4. Overriding and super

You can change the property of an object any time, just by assigning something to it. What about a class, though? Assigning a new value to a class property works the same as any object:

```
class Human

Human.rights = ['Life', 'Liberty', 'the pursuit of happiness']
Human.rights
# ['Life', 'Liberty', 'the pursuit of happiness']

Human.rights = Human.rights.concat ['To party']
Human.rights
# ['Life', 'Liberty', 'the pursuit of happiness', 'To party']
```

Class methods are just properties on a class. How about instance methods? What happens when you write a class declaration that defines an instance method that's already inherited from another class through `extends`?

5.4.1. Overriding

Imagine now that Agtron wants cameras to look different than other products on the shop website. He wants them to have a photo gallery next to their product description. The `Gallery` class has a `constructor` and a `render` method:

```
class Gallery
  constructor: ->
  render: ->
```

Using the `constructor`, a new gallery is created with each new camera:

```
class Camera
  constructor: ->
    @gallery = new Gallery
```

When the camera renders, it will need to invoke the `render` method on the gallery. This means the `Camera` class needs a different `render` method from other products. Put this method inside the class declaration: